

Robustness of Non-negative Matrix Factorization under Contiguous Occlusion Noise

COMP4328/5328/8328 Advanced Machine Learning — Assignment 1

Group ID: **XX**

Name: Zihan Li; SID: 550521804; unikey: zili0238

Name: Runqi Dai; SID: 550429256; unikey: rdai0696

Name: Elwood Fong; SID: 510444907; unikey: efon8141

Name 4 SID 4 unikey 4

Abstract

Non-negative Matrix Factorization (NMF) learns a parts-based, additive representation of non-negative data, but its standard Frobenius-norm formulation is a least-squares estimator and therefore has an unbounded influence function: a handful of grossly corrupted pixels can dominate the objective and destroy the learned basis. This report studies how far that weakness can be repaired by changing the loss. We implement five NMF algorithms from scratch in NumPy — the Frobenius-norm multiplicative-update baseline, L_1 -norm NMF, hypersurface-cost NMF, $L_{2,1}$ -norm NMF, and an L_1 -regularised robust NMF with an explicit sparse outlier term — and evaluate them on the ORL and Extended YaleB face datasets contaminated by contiguous occlusion blocks. We vary three independent properties of the corruption: the block edge length, the number of blocks per image, and the *proportion of images* corrupted, and measure relative reconstruction error, clustering accuracy and normalised mutual information over five independent 90% subsamples. The sparse-outlier model cuts RRE by 52% relative to the Frobenius baseline at $b = 15$ on Extended YaleB (0.323 against 0.668), while $L_{2,1}$ -NMF is numerically indistinguishable from that same baseline — agreeing to three decimal places — on every condition in which all images are corrupted equally, and separates from it only once the corruption is unevenly distributed across samples. Section 1 introduces the problem, Section 2 reviews related NMF variants, Section 3 derives the five algorithms and the noise model, Section 4 reports the experiments, and Section 5 concludes.

1 Introduction

1.1 Non-negative matrix factorization

Given a non-negative data matrix $\mathbf{V} \in \mathbb{R}_+^{m \times n}$ whose n columns are vectorised images of m pixels each, NMF seeks factors $\mathbf{W} \in \mathbb{R}_+^{m \times k}$ and $\mathbf{H} \in \mathbb{R}_+^{k \times n}$ with $k \ll \min(m, n)$ such that $\mathbf{V} \approx \mathbf{WH}$. The two factors admit a direct reading: the k columns of $\mathbf{W}$ are themselves images and act as a learned dictionary of *basis images*, while column j of $\mathbf{H}$ holds the k coefficients that reconstruct image j as the additive mixture $\mathbf{v}_j \approx \sum_{t=1}^k \mathbf{w}_t H_{tj}$.

What distinguishes NMF from the other classical low-rank factorisations is precisely the constraint that makes it harder to solve. Principal component analysis minimises the same reconstruction error over unconstrained factors, so its components may take either sign and a reconstruction may *cancel*: a bright region in one component can be removed by a negative coefficient on another. The individual components are consequently holistic — eigenfaces look like whole, ghostly faces — and are not interpretable as parts. Forbidding negative entries forbids cancellation outright. Every basis image can only ever be *added* to the reconstruction, so the only way to represent a set of faces with few components is to discover pieces that actually recur across them, which is why NMF bases for face data tend to localise onto eyes, noses, mouths and hairlines [11]. The same argument explains the breadth of its applications: topics as additive mixtures of words in document

collections, endmembers in hyperspectral unmixing, spectrogram atoms in audio source separation, and latent factors in recommendation, all of which are domains where the data are non-negative and a subtractive explanation would be physically meaningless.

1.2 The problem: robustness to gross corruption

The standard NMF objective is the squared Frobenius error $\|\mathbf{V} - \mathbf{WH}\|_F^2$. Minimising it is maximum-likelihood estimation under the assumption that every entry of $\mathbf{V}$ is the true value plus independent Gaussian noise of a common variance. That assumption is a statement about the *whole* data matrix, and real face imagery violates it in a specific and damaging way. Sunglasses, scarves, a hand in front of a face, specular highlights, cast shadows, compression blocks and dead sensor pixels do not perturb many pixels slightly; they replace a *few* pixels *completely*. The corruption is sparse in its support but unbounded in magnitude — the opposite of the Gaussian regime.

Under a squared loss the consequences are not merely a loss of accuracy but a loss of the estimator itself. The influence function of the least-squares loss, $\psi(r) = \partial_r r^2 = 2r$, is unbounded, so the contribution of a single residual to the gradient grows without limit as that residual grows. An occlusion block of edge length b filled with the saturated value contributes on the order of b^2 squared residuals of near-maximal size, and the optimiser will happily sacrifice the fit on thousands of clean pixels to reduce them. In the language of robust statistics the breakdown point is zero: an arbitrarily small fraction of arbitrarily corrupted entries suffices to drive the estimate arbitrarily far from the truth. Empirically this shows up as bright, square-shaped artefacts absorbed directly into the learned basis images, which is visible in Section 4.7.

Two families of remedy exist. One may *change the loss* so that the influence of a large residual is bounded — replacing the square by an absolute value (L_1), by a smooth interpolation between the two (hypersurface / pseudo-Huber), or by a per-column norm that down-weights whole corrupted samples ($L_{2,1}$). Alternatively one may *model the corruption explicitly*, adding a sparse outlier matrix that absorbs the occlusion so that the low-rank part never has to explain it. This report asks a concrete question about these remedies: given contiguous occlusion noise of a controlled size, count and prevalence, which of them actually recovers the clean signal, by how much, and — the question that turns out to matter most — *under which corruption geometry* does each one help at all?

1.3 Overview of the approach

The five algorithms differ in the granularity at which they respond to a large residual: Frobenius NMF is the least-squares baseline; L_1 -NMF and hypersurface-cost NMF down-weight pixels; $L_{2,1}$ -NMF down-weights images; and the robust model gives corruption its own sparse outlier variable. We test whether robustness helps when this granularity matches the corruption pattern. Four algorithms extend beyond the course methods; Frobenius NMF is the taught baseline.

1.4 Contributions

This report makes the following contributions.

1. We implement five NMF algorithms from scratch using NumPy and the Python standard library, and derive them from one majorise–minimise (MM) template, isolating the granularity at which each down-weights a residual (Section 3).
2. We show that a column-wise robust loss such as $L_{2,1}$ can distinguish clean from corrupted samples only when corruption is *unevenly distributed across samples*; hence contamination ratio is a third noise axis (Section 3.8).

3. We report a rigorous evaluation — three metrics, five independent 90% subsamples, and an explicit convergence study — on both required datasets (Section 4).
4. We show that the hypersurface cost’s published unit scale is almost quadratic on data normalised to $[0, 1]$, so its scale parameter must be chosen relative to the pixel range rather than copied unchanged.
5. We use sensitivity sweeps to show that the main conclusions survive reasonable choices of the smoothing, hypersurface-scale, outlier- penalty and rank parameters.

2 Related work

2.1 Classical NMF

Paatero and Tapper [15] introduced positive matrix factorization, and Lee and Seung [11] popularised NMF together with the multiplicative update (MU) rules that make the non-negativity constraint free of charge. Their argument is an auxiliary-function (majorise–minimise) one: for the current iterate one constructs a surrogate $G(\mathbf{H}, \mathbf{H}^t)$ that lies above the objective everywhere and touches it at $\mathbf{H}^t$, and then minimises the surrogate in closed form. Because $F(\mathbf{H}^{t+1}) \leq G(\mathbf{H}^{t+1}, \mathbf{H}^t) \leq G(\mathbf{H}^t, \mathbf{H}^t) = F(\mathbf{H}^t)$, the objective is non-increasing at every step by construction, and because the update is a *ratio* of non-negative quantities the iterates stay feasible without any projection or step-size search. Lee and Seung [12] give the same treatment for the Kullback–Leibler divergence.

Advantages. No step size to tune, no constraint to enforce, $O(mnk)$ per update, and monotone descent that makes the method trivial to debug. *Disadvantages.* Monotone descent is weaker than convergence: Lin [14] showed the original rules need not reach a stationary point, and the tail convergence is slow. More importantly here, both classical costs are maximum-likelihood estimators for a *specific* noise model — Gaussian for the Frobenius norm, Poisson for KL — and both give large residuals unbounded influence.

2.2 Robust loss functions

The first family keeps the factorisation model and replaces the discrepancy measure.

L_1 -norm NMF. Ke and Kanade [9] and Ding et al. [4] replace the squared error by the entrywise L_1 norm, the maximum-likelihood criterion under Laplacian noise. Its influence function $\text{sign}(r)$ is bounded, so a pixel off by 0.9 pulls no harder than one off by 0.1 and the fit can abandon occluded pixels instead of chasing them. *Advantage:* robustness at the granularity of the individual pixel, which matches the support of occlusion noise exactly. *Disadvantage:* non-differentiable at zero, and the reweighted surrogate’s $1/|r|$ weights blow up on the well-fitted pixels, so a smoothing constant δ must be introduced — a genuine hyper-parameter (Section 4.9).

Hypersurface cost. Hamza and Brady [7] propose the pseudo-Huber loss $\phi(r) = \sqrt{1 + r^2} - 1$, quadratic for $|r| \ll 1$ and linear for $|r| \gg 1$. *Advantage:* differentiable everywhere, so no smoothing constant is needed, and it combines quadratic efficiency on inliers with L_1 ’s bounded influence on outliers. *Disadvantage:* the crossover sits at $|r| = 1$ in the units of the data, so with pixels in $[0, 1]$ every residual stays in the quadratic regime and the method silently degenerates into the Frobenius baseline unless a scale constant c is introduced — a trap we examine in Section 3.5.

$L_{2,1}$ -norm NMF. Kong et al. [10] use $\|\mathbf{R}\|_{2,1} = \sum_j \|\mathbf{r}_j\|_2$, the sum over columns of the per-column Euclidean norm. Each column is one image, so the cost grows only *linearly* in the size of an image’s residual, and the induced reweighting $d_j = 1/\|\mathbf{r}_j\|_2$ acts as a soft sample-selection mechanism that

discounts badly damaged images as a whole. *Advantage*: the right model when entire observations are unreliable — a corrupted photograph, a mislabelled document, a faulty sensor channel. *Disadvantage, and the one that drives the experimental design of this report*: the robustness is only at the sample level. If *every* column is corrupted to the same degree, all d_j are nearly equal, a uniform column weighting is absorbed by the scaling freedom of the factorisation, and $L_{2,1}$ -NMF reduces numerically to plain Frobenius NMF. It cannot see *which pixels* inside a column are bad, only *which columns* are worse than the others.

2.3 Explicit outlier models

A second family leaves the loss quadratic and instead enlarges the model. Robust PCA [3] decomposes $\mathbf{V} = \mathbf{L} + \mathbf{S}$ into a low-rank term and a sparse term and recovers both exactly, under incoherence conditions, by minimising $\|\mathbf{L}\|_* + \lambda \|\mathbf{S}\|_1$. Zhang et al. [17] carry the same idea into the non-negative setting, writing $\mathbf{V} \approx \mathbf{W}\mathbf{H} + \mathbf{S}$ with an L_1 penalty on $\mathbf{S}$ and solving by alternating soft-thresholding of $\mathbf{S}$ with multiplicative updates of $\mathbf{W}, \mathbf{H}$; Ding et al. [5] give a related non-negative treatment.

Advantages. The corruption is *explained* rather than discounted: the low-rank part is never asked to represent the occlusion, which is the right prior for a contiguous white block, and the recovered $\mathbf{S}$ is an interpretable occlusion map. The remaining loss is quadratic, so Lee–Seung applies unchanged to $\mathbf{W}$ and $\mathbf{H}$. *Disadvantages*. An extra $m \times n$ variable doubles the memory footprint, and the penalty λ trades reconstruction against sparsity — too small and $\mathbf{S}$ absorbs the signal, too large and it stays empty (Section 4.9).

2.4 Other directions

A third line of work adds *structure* to the representation rather than robustness to the loss: graph-regularised NMF [2], sparse NMF [8], orthogonal NMF [4] and deep multi-layer NMF. These are complementary to the methods studied here — they change what a *good* factorisation looks like, whereas we ask whether the factorisation survives grossly corrupted inputs at all. We therefore hold the representation-side choices fixed (rank k equal to the class count, no extra regularisation) so that the only variable across the five algorithms is how each treats a large residual.

3 Methods

3.1 Notation and pre-processing

Each image is converted to 8-bit greyscale, downsampled by an integer factor (3 for ORL, giving 37×30 pixels; 4 for Extended YaleB, giving 48×42), vectorised column-wise, and divided by 255 so that every entry lies in $[0, 1]$. Downsampling is the only lossy step and is applied for tractability: it reduces the per-iteration cost of the multiplicative updates, which is $O(mnk)$. The resulting data matrices are $\mathbf{V} \in \mathbb{R}_+^{1110 \times 400}$ for ORL (40 subjects, 10 images each) and $\mathbf{V} \in \mathbb{R}_+^{2016 \times 2414}$ for Extended YaleB (38 subjects; the per-folder *Ambient.pgm frames, which are illumination references rather than face captures, are excluded by the loader). The rank is set to the number of classes present in the subsample, $k = 40$ and $k = 38$ respectively, so that the comparison is never confounded by a difference in model capacity.

Deliberately, *no* further pre-processing is applied. Mean-centring and whitening — both standard before PCA — are inadmissible here: each would introduce negative entries and so violate the constraint that defines the model. We likewise do not contrast-normalise the Extended YaleB images, even though their illumination varies by orders of magnitude, because doing so would suppress exactly the kind of large-magnitude structured variation whose effect on the factorisation this report sets out to measure. The only operations applied to the data are therefore greyscale conversion,

Table 1: The five algorithms as instances of the same MM template. $\mathbf{R} = \mathbf{V} - \mathbf{WH}$ is the current residual.

Algorithm	Objective	Weight $\mathbf{D}$	Down-weights at
F-norm MU	$\ \mathbf{R}\ _F^2$	1	— (baseline)
L_1 -NMF	$\ \mathbf{R}\ _1$	$1/(R_{ij} + \delta)$	pixel
Hypersurface	$\sum_{ij} (\sqrt{1 + R_{ij}^2} - 1)$	$1/\sqrt{1 + R_{ij}^2}$	pixel
$L_{2,1}$ -NMF	$\ \mathbf{R}\ _{2,1}$	$1/\ \mathbf{R}_{:,j}\ _2$	image
L_1 -Reg Robust	$\frac{1}{2} \ \mathbf{V} - \mathbf{WH} - \mathbf{S}\ _F^2 + \lambda \ \mathbf{S}\ _1$	explicit outlier variable $\mathbf{S}$	

downsampling and a global division by 255, all of which are order-preserving and non-negativity-preserving.

3.2 A single majorise–minimise template

All algorithms except the sparse-outlier model of Section 3.7 minimise a separable cost of the residual $\mathbf{R} = \mathbf{V} - \mathbf{WH}$,

$$E(\mathbf{W}, \mathbf{H}) = \sum_{ij} \phi(R_{ij}), \quad \mathbf{W}, \mathbf{H} \geq 0, \quad (1)$$

for a function ϕ that is concave and increasing in r^2 . For any such ϕ , concavity gives the tangent upper bound

$$\phi(r) \leq \phi(r^t) + \underbrace{\frac{\phi'(r^t)}{2r^t}}_{=D^t} (r^2 - (r^t)^2), \quad (2)$$

which is a *weighted quadratic* that touches ϕ at the current iterate. Minimising the resulting weighted Frobenius surrogate with Lee–Seung preconditioning yields the multiplicative updates

$$\mathbf{W} \leftarrow \mathbf{W} \odot \frac{(\mathbf{D} \odot \mathbf{V}) \mathbf{H}^\top}{(\mathbf{D} \odot \mathbf{WH}) \mathbf{H}^\top}, \quad \mathbf{H} \leftarrow \mathbf{H} \odot \frac{\mathbf{W}^\top (\mathbf{D} \odot \mathbf{V})}{\mathbf{W}^\top (\mathbf{D} \odot \mathbf{WH})}, \quad (3)$$

where $\odot$ and the fraction bar are elementwise and $\mathbf{D}$ is held fixed within an iteration and recomputed from the current residual at the start of the next one (an iteratively reweighted least squares, or IRLS, scheme). Because both updates are multiplicative and all factors are non-negative, $\mathbf{W}, \mathbf{H} \geq 0$ is maintained without any projection step.

The practical consequence is that **the five algorithms differ only in $\mathbf{D}$** , as summarised in Table 1. This is what makes the comparison in Section 4 a controlled experiment rather than a comparison of five different codebases.

3.3 Algorithm 1: Frobenius-norm NMF (baseline, taught in this course)

$$\min_{\mathbf{W}, \mathbf{H} \geq 0} \|\mathbf{V} - \mathbf{WH}\|_F^2, \quad \phi(r) = r^2, \quad D_{ij} = 1. \quad (4)$$

With $\mathbf{D} = \mathbf{1}$ the template of Eq. (3) collapses to the classical Lee–Seung multiplicative updates

$$\mathbf{W} \leftarrow \mathbf{W} \odot \frac{\mathbf{V} \mathbf{H}^\top}{\mathbf{W} \mathbf{H} \mathbf{H}^\top}, \quad \mathbf{H} \leftarrow \mathbf{H} \odot \frac{\mathbf{W}^\top \mathbf{V}}{\mathbf{W}^\top \mathbf{W} \mathbf{H}}. \quad (5)$$

Monotonicity. For fixed $\mathbf{W}$, the function $G(\mathbf{H}, \mathbf{H}^t) = F(\mathbf{H}^t) + (\mathbf{H} - \mathbf{H}^t)^\top \nabla F(\mathbf{H}^t) + \frac{1}{2} (\mathbf{H} - \mathbf{H}^t)^\top \mathbf{K}(\mathbf{H}^t) (\mathbf{H} - \mathbf{H}^t)$, with the diagonal matrix $K_{ab}(\mathbf{H}^t) = \delta_{ab} (\mathbf{W}^\top \mathbf{W} \mathbf{H}^t)_a / H_a^t$, majorises F and touches it at $\mathbf{H}^t$, because $\mathbf{K}(\mathbf{H}^t) - \mathbf{W}^\top \mathbf{W} \succeq 0$ [12]. Setting $\nabla_{\mathbf{H}} G = 0$ yields precisely the update above, so $F(\mathbf{H}^{t+1}) \leq G(\mathbf{H}^{t+1}, \mathbf{H}^t) \leq G(\mathbf{H}^t, \mathbf{H}^t) = F(\mathbf{H}^t)$: the objective never increases, and since it is bounded below by zero the sequence of objective values converges. The same argument applies to $\mathbf{W}$ by symmetry.

Theoretical robustness: none. The influence of a residual is $\phi'(r) = 2r$, which is *unbounded*. A single saturated pixel therefore exerts arbitrarily large leverage on the gradient, and the estimator has a breakdown point of zero. Concretely, a block of edge length b filled with 1.0 contributes $O(b^2)$ residuals of near-maximal magnitude, and since each is squared the optimiser will trade away the fit on many clean pixels to reduce them — which is why the baseline’s learned basis absorbs the occlusion outright (Section 4.7). Every other algorithm below is a way of making ϕ' bounded, or of removing the offending residuals from the loss altogether.

3.4 Algorithm 2: L_1 -norm NMF (not taught)

$$\min_{\mathbf{W}, \mathbf{H} \geq 0} \|\mathbf{V} - \mathbf{WH}\|_1 = \sum_{ij} |V_{ij} - (\mathbf{WH})_{ij}|. \quad (6)$$

Majorising $|r| \leq r^2/(2s) + s/2$ at $s = |r^f|$ gives $D_{ij} = 1/|R_{ij}|$, i.e. an iteratively-reweighted least-squares scheme in which each pixel is weighted by the reciprocal of its own current error.

Theoretical robustness. The influence function is $\phi'(r) = \text{sign}(r)$, bounded by 1 regardless of $|r|$. A pixel that is wrong by 0.9 pulls on the solution no harder than one wrong by 0.1, so the fit is free to abandon the occluded pixels rather than chase them. Because the weighting is *entrywise*, the robustness operates at exactly the granularity of the corruption: occlusion is sparse in the pixel domain, and L_1 -NMF discounts pixels.

The smoothing constant is not cosmetic. The weight $1/|R_{ij}|$ diverges as a residual approaches zero — which is what happens on the well-fitted majority of pixels — so in practice one uses $D_{ij} = 1/(|R_{ij}| + \delta)$. This is not a numerical guard bolted onto the L_1 scheme; it is exactly the IRLS weighting of the *smoothed* loss $\sqrt{r^2 + \delta^2}$, so δ sets the residual scale below which the loss is treated as quadratic. With pixels in $[0, 1]$, our default $\delta = 10^{-3}$ declares residuals under one tenth of one percent of the dynamic range to be noise-free. Because this is a modelling choice rather than a free knob, Section 4.9 sweeps δ over four orders of magnitude to confirm that the conclusions do not rest on the particular value.

3.5 Algorithm 3: hypersurface-cost NMF (not taught)

$$\min_{\mathbf{W}, \mathbf{H} \geq 0} \sum_{ij} \left(\sqrt{1 + (V_{ij} - (\mathbf{WH})_{ij})^2} - 1 \right), \quad D_{ij} = \frac{1}{\sqrt{1 + R_{ij}^2}}. \quad (7)$$

A Taylor expansion shows what this loss does: $\phi(r) \approx r^2/2$ for $|r| \ll 1$ and $\phi(r) \approx |r|$ for $|r| \gg 1$. It is thus quadratic — and therefore statistically efficient — on the clean pixels, and linear — and therefore robust — on the corrupted ones, with a smooth transition between the two. Its influence function $\phi'(r) = r/\sqrt{1 + r^2}$ is bounded by 1, as for L_1 , but unlike L_1 it is continuous at the origin, so the loss is differentiable everywhere and needs *no* hand-chosen smoothing constant to be well posed [7]. Its weights lie in $(0, 1]$ rather than being unbounded, which makes the reweighting markedly less aggressive than the L_1 scheme.

The scale constant, and a trap. The transition between the two regimes sits at $|r| = 1$ in the units in which the cost is written. Our pixels live in $[0, 1]$, so *every attainable residual* satisfies $|r| \leq 1$ and the method never leaves its quadratic regime: with the literature’s $c = 1$ the hypersurface cost silently degenerates into the Frobenius baseline, and its “robustness” is an illusion. We therefore implement the scaled cost $\phi_c(r) = c^2(\sqrt{1 + (r/c)^2} - 1)$ with weights $D_{ij} = (1 + (R_{ij}/c)^2)^{-1/2}$ and default $c = 0.05$, placing the elbow at 5% of the dynamic range so that clean pixels remain in the quadratic regime while occluded ones (with r of order 1) are firmly in the linear one. Section 4.9 sweeps c and shows the degeneration happening as $c \rightarrow 1$.

3.6 Algorithm 4: $L_{2,1}$ -norm NMF (not taught)

$$\min_{\mathbf{W}, \mathbf{H} \geq 0} \|\mathbf{V} - \mathbf{WH}\|_{2,1} = \sum_{j=1}^n \sqrt{\sum_{i=1}^m (V_{ij} - (\mathbf{WH})_{ij})^2}, \quad d_j = \frac{1}{\|\mathbf{R}_{:,j}\|_2}. \quad (8)$$

Since $\|\mathbf{r}\|_2 = \min_{d>0} (\frac{d}{2}\|\mathbf{r}\|_2^2 + \frac{1}{2d})$ with optimum $d = 1/\|\mathbf{r}\|_2$, the surrogate is a *column*-weighted Frobenius cost and Eq. (3) applies with $\mathbf{D} = \mathbf{1}\mathbf{d}^\top$.

A prediction we test explicitly. Because d_j varies only *across* columns, $L_{2,1}$ -NMF performs soft sample selection: it can discount an image, never a pixel. If every image receives the same amount of occlusion, all $\|\mathbf{R}_{:,j}\|_2$ are nearly equal, $\mathbf{d}$ is nearly constant, and a constant column weighting is absorbed by the scale freedom of the factorisation — so $L_{2,1}$ -NMF should then be numerically *indistinguishable* from the Frobenius baseline. This prediction is the reason we sweep the contamination ratio in Section 3.8, and Section 4.5 reports the test.

3.7 Algorithm 5: L_1 -regularised robust NMF (not taught)

$$\min_{\mathbf{W}, \mathbf{H} \geq 0, \mathbf{S}} \frac{1}{2} \|\mathbf{V} - \mathbf{WH} - \mathbf{S}\|_F^2 + \lambda \|\mathbf{S}\|_1. \quad (9)$$

Rather than down-weighting corrupted entries, this model gives the corruption its own variable: $\mathbf{S}$ is an explicit sparse outlier matrix, so $\mathbf{WH}$ is fitted to the *cleaned* data $\mathbf{V} - \mathbf{S}$. Block-coordinate descent alternates a closed-form soft-thresholding step and a Frobenius MU step:

$$\mathbf{S} \leftarrow \mathcal{S}_\lambda(\mathbf{V} - \mathbf{WH}), \quad \mathcal{S}_\lambda(x) = \text{sign}(x) \max(|x| - \lambda, 0), \quad (10)$$

$$\mathbf{W} \leftarrow \mathbf{W} \odot \frac{\mathbf{TH}^\top}{\mathbf{WHH}^\top}, \quad \mathbf{H} \leftarrow \mathbf{H} \odot \frac{\mathbf{W}^\top \mathbf{T}}{\mathbf{W}^\top \mathbf{WH}}, \quad \mathbf{T} = \max(\mathbf{V} - \mathbf{S}, 0). \quad (11)$$

Descent. Eq. (10) is the exact minimiser of the joint objective in $\mathbf{S}$ for fixed $(\mathbf{W}, \mathbf{H})$ — the proximal operator of $\lambda \|\cdot\|_1$ — and Eq. (11) is a Lee–Seung step on the quadratic term with $\mathbf{S}$ fixed, which is non-increasing by the auxiliary-function argument of Section 3.3. Each block step therefore leaves the joint objective non-increasing, and since it is bounded below by 0 the sequence of objective values converges. (As with all block-coordinate schemes on a non-convex problem, this guarantees convergence of the objective, not of the iterates to a global minimum.)

Choosing λ . We use the standard Robust-PCA scaling $\lambda = 1/\sqrt{\max(m, n)}$ [3], which is the value for which exact recovery of a sparse-plus-low-rank decomposition holds under incoherence. It has an intuitive reading as a threshold: a residual must exceed λ before it is declared an outlier and moved into $\mathbf{S}$. The sweep in Section 4.9 varies λ by two orders of magnitude around this value, and shows the two failure modes at the ends of the range — $\mathbf{S}$ absorbing genuine signal when λ is too small, and $\mathbf{S}$ staying empty (so that the method reduces to the Frobenius baseline) when it is too large.

Why this model should fit our noise best. A white occlusion block is *precisely* a sparse, large-magnitude, additive perturbation: few entries affected, each affected entry changed by an amount of order the full dynamic range. That is the generative assumption of $\mathbf{V} = \mathbf{WH} + \mathbf{S}$ stated exactly. The other four algorithms can only *discount* corrupted entries, leaving a biased residual in the loss; this one *removes* them from the quadratic term altogether, and returns the estimated occlusion map $\mathbf{S}$ as a by-product that can be inspected directly. We should therefore expect it to lead on the heavy-occlusion conditions, and Section 4 confirms that it does.

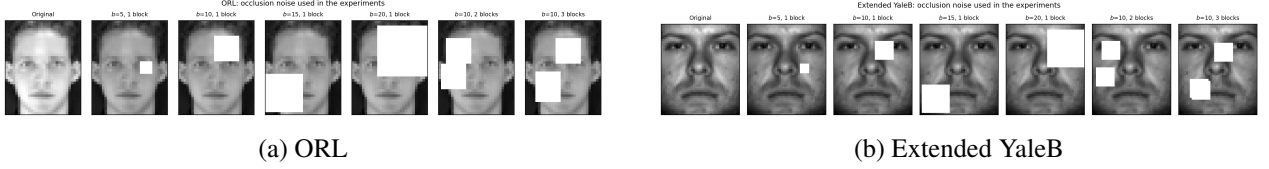


Figure 1: Demonstration of the noise: an original image and the same image under the occlusion conditions used in the experiments.

3.8 Noise model

Following the brief we use *contiguous occlusion*: squares of edge length b filled with the saturated value 255 (i.e. 1.0 after normalisation) are pasted at uniformly random positions. We control three independent properties of the corruption:

1. the **block edge length** $b \in \{0, 5, 10, 15, 20\}$ pixels ($b = 0$ is the uncorrupted control);
2. the **number of blocks** per image $\in \{0, 1, 2, 3, 4\}$ at $b = 10$;
3. the **contamination ratio**, i.e. the fraction of images that are corrupted at all, $\in \{0.1, 0.25, 0.5, 0.75, 1.0\}$ at $b = 15$ with two blocks per corrupted image.

The first two axes are named in the brief. The third is ours, and it is not cosmetic: axes 1–2 change *how many pixels* are corrupted while keeping every image equally damaged, whereas axis 3 changes *how many images* are corrupted. A per-pixel robust loss (L_1 , hypersurface) responds to the former; a per-image robust loss ($L_{2,1}$) can only respond to the latter. Sweeping only axes 1–2 would therefore make $L_{2,1}$ -NMF look identical to the non-robust baseline and would hide the very phenomenon the assignment asks us to demonstrate.

Because blocks are placed independently they may overlap, so the nominal budget $n_{\text{blocks}} \cdot b^2$ overstates the damage; we therefore log the *measured* fraction of modified entries for every condition and report it alongside the nominal settings.

3.9 Evaluation metrics

Let $\hat{\mathbf{V}}$ be the clean data and $\mathbf{W}, \mathbf{H}$ the factors fitted on the *contaminated* matrix $\mathbf{V}$. The required metric is the relative reconstruction error

$$\text{RRE} = \frac{\|\hat{\mathbf{V}} - \mathbf{W}\mathbf{H}\|_F}{\|\hat{\mathbf{V}}\|_F}. \quad (12)$$

Note the asymmetry: the algorithm sees only $\mathbf{V}$ but is scored against $\hat{\mathbf{V}}$, so fitting the occlusion *well* makes the score *worse*. We additionally report the two optional metrics, clustering accuracy and normalised mutual information, obtained by running K -means with K equal to the number of classes on the columns of $\mathbf{H}$ and mapping clusters to labels by majority vote, as in the provided tutorial notebook. We also log an accuracy computed with optimal one-to-one (Hungarian) matching, which cannot be inflated by several clusters collapsing onto the same class.

Protocol. Every experiment draws an independent uniform 90% subsample of the images, and is repeated 5 times; we report the mean and the sample standard deviation across those repeats. Within a repeat, all noise conditions and all algorithms see the identical subsample, and all algorithms see the identical corrupted matrix, so differences between algorithms are not confounded by the draw. The rank is fixed to $k =$ the number of classes present in the subsample.

Table 2: Representative realised fractions of modified pixels, measured from the generated masks; overlap can reduce them below nominal block area.

Condition	ORL	Extended YaleB
$b = 5$, 1 block	2.3%	1.2%
$b = 10$, 1 block	9.0%	5.0%
$b = 15$, 1 block	20.3%	11.1%
$b = 20$, 1 block	36.0%	19.8%
$b = 10$, 2 blocks	16.9%	9.6%
$b = 10$, 4 blocks	29.8%	18.0%
$b = 15$, 2 blocks	34.0%	20.4%

Table 3: Datasets after pre-processing.

Dataset	Images	Subjects	Native size	Reduce	V
ORL	400	40	112×92	3	1110×400
Extended YaleB	2414	38	192×168	4	2016×2414

4 Experiments and discussion

4.1 Experimental setup

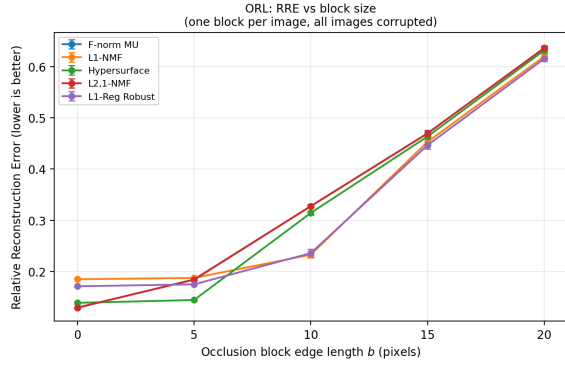
Every experiment factorises a contaminated matrix and scores the result against the *clean* one. The two datasets are summarised in Table 3; all five algorithms of Section 3 are run on every condition, so each cell of every table below is a like-for-like comparison in which the only thing that differs between rows is the loss.

Noise grid. Three axes are swept independently, each around a common reference point. *Block size*: $b \in \{0, 5, 10, 15, 20\}$ pixels with one block per image, where $b = 0$ is the uncontaminated control. *Block count*: $\{0, 1, 2, 3, 4\}$ blocks per image at a fixed $b = 10$. *Contamination ratio*: the fraction of images corrupted, $\{0.1, 0.25, 0.5, 0.75, 1.0\}$, at a fixed $b = 15$ with 2 blocks per corrupted image. Blocks are square, filled with the saturated value 1.0, placed uniformly at random, and allowed to overlap.

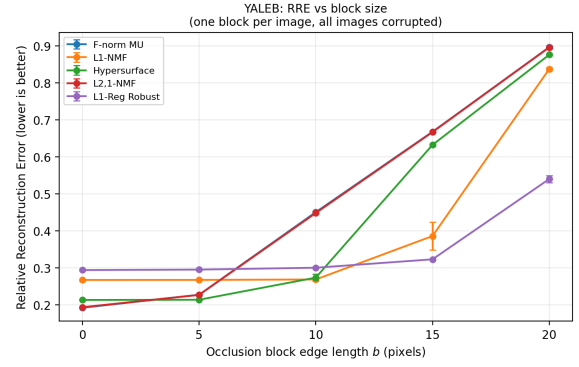
Datasets. We use the ORL face database [16] and Extended YaleB [6, 13]; their pre-processed dimensions are shown in Table 3.

Protocol. Following the note in the brief, every condition is repeated over $R = 5$ independent random subsamples of 90% of the images, and we report mean and standard deviation across those repeats. The subsample for repeat r is drawn once and *shared by all conditions and all algorithms* in that repeat, so differences between algorithms are never confounded by differences in the data they saw. The rank is $k =$ the number of classes present in the subsample (40 for ORL, 38 for YaleB). Multiplicative updates run for at most 400 iterations on ORL and 250 on YaleB, with an early stop when the relative change in the objective falls below $\tau = 10^{-5}$; the convergence study in Section 4.8 justifies these budgets. The subsample and factor initialisation are seeded by repeat, while the noise seed is a deterministic function of the repeat and noise condition. Thus every condition within a repeat starts from the same factors and uses the same reproducible noise construction, and the entire grid is reproducible with `bash run_all.sh`.

Metrics. All three metrics of Section 3.9 — RRE (required), clustering accuracy and NMI (both optional) — are reported for every condition. Timings indicate relative cost between algorithms rather than absolute performance.



(a) ORL



(b) Extended YaleB

Figure 2: RRE against occlusion block edge length b (one block per image, all images corrupted). Error bars are one standard deviation over five 90% subsamples.

Table 4: ORL: RRE (lower is better) against block edge length b . Mean $\pm$ standard deviation over five independent 90% subsamples; best per column in bold.

Algorithm	0	5	10	15	20
F-norm MU	0.1290 ± 0.0005	0.1846 ± 0.0009	0.3277 ± 0.0032	0.4699 ± 0.0056	0.6363 ± 0.0045
L1-NMF	0.1849 ± 0.0012	0.1874 ± 0.0009	0.2328 ± 0.0065	0.4530 ± 0.0071	0.6200 ± 0.0046
Hypersurface	0.1388 ± 0.0007	0.1443 ± 0.0010	0.3143 ± 0.0043	0.4636 ± 0.0068	0.6319 ± 0.0048
L2,1-NMF	0.1295 ± 0.0007	0.1840 ± 0.0007	0.3274 ± 0.0032	0.4696 ± 0.0058	0.6364 ± 0.0045
L1-Reg Robust	0.1711 ± 0.0012	0.1749 ± 0.0007	0.2358 ± 0.0077	0.4466 ± 0.0076	0.6154 ± 0.0051

4.2 Sanity check: the uncontaminated control

Before any robustness claim can be made, the five algorithms must be shown to agree when there is nothing to be robust to; a large gap at $b = 0$ would indicate an implementation or convergence fault rather than a property of the loss. On clean ORL the Frobenius baseline attains $\text{RRE} = 0.1290 \pm 0.0005$ and $L_{2,1}$ -NMF 0.1295 ± 0.0007 — indistinguishable, as Section 3 predicts they must be when no column is worse than any other. Hypersurface-cost NMF is close behind at 0.1388, while L_1 -NMF (0.1849) and the sparse-outlier model (0.1711) are 43% and 33% worse respectively. The clustering metrics order the five the same way (accuracy 0.760 for the baseline against 0.641 for L_1), which confirms the ordering is a property of the fits and not an artefact of the reconstruction metric.

That the robust losses are *worse* on clean data is the usual efficiency/robustness trade-off. Each achieves bounded influence by discounting large residuals, and on clean data a large residual is not corruption but signal — a hard-to-reconstruct region of a face. Hypersurface-cost NMF pays the smallest price because its weights are bounded in $(0, 1]$, so its reweighting is the gentlest. This row is the correct baseline for every later table: a robust method must first recover this deficit before showing any net benefit.

Extended YaleB reproduces the pattern with the gaps widened. The baseline attains 0.1917 ± 0.0009 and $L_{2,1}$ -NMF 0.1932 ± 0.0005 — again within a thousandth of each other — while L_1 -NMF (0.2668) and the sparse-outlier model (0.2935) are 39% and 53% worse. The larger deficit is expected here: YaleB’s extreme illumination variation produces genuinely large residuals in shadowed regions, and a robust loss cannot distinguish “large because corrupted” from “large because dark”. Clustering scores on YaleB are low throughout (best accuracy 0.240, against 0.760 on ORL) for the same reason — a K -means partition of YaleB codes groups by illumination rather than identity.

Table 5: Extended YaleB: RRE (lower is better) against block edge length b . Mean $\pm$ standard deviation over five independent 90% subsamples; best per column in bold.

Algorithm	0	5	10	15	20
F-norm MU	0.1917 $\pm$ 0.0009	0.2270 $\pm$ 0.0024	0.4502 $\pm$ 0.0019	0.6675 $\pm$ 0.0023	0.8962 $\pm$ 0.0039
L1-NMF	0.2668 $\pm$ 0.0026	0.2670 $\pm$ 0.0026	0.2682 $\pm$ 0.0026	0.3856 $\pm$ 0.0377	0.8370 $\pm$ 0.0043
Hypersurface	0.2128 $\pm$ 0.0011	0.2134 $\pm$ 0.0009	0.2732 $\pm$ 0.0096	0.6320 $\pm$ 0.0029	0.8756 $\pm$ 0.0021
L2,1-NMF	0.1932 $\pm$ 0.0005	0.2265 $\pm$ 0.0020	0.4478 $\pm$ 0.0019	0.6665 $\pm$ 0.0013	0.8955 $\pm$ 0.0039
L1-Reg Robust	0.2935 $\pm$ 0.0016	0.2949 $\pm$ 0.0017	0.3000 $\pm$ 0.0019	0.3227 $\pm$ 0.0018	0.5396 $\pm$ 0.0091

4.3 Effect of occlusion size

RRE grows monotonically with b for all five algorithms, but not at the same rate, and the ordering changes twice as b increases.

At $b = 5$ (2.3% of ORL pixels) only the hypersurface cost improves on the baseline, 0.1443 against 0.1846, a 22% reduction. L_1 -NMF sits at 0.1874, no better than the baseline — but it has degraded by 1% from its own $b = 0$ value while the baseline degraded by 43%. It is not yet *ahead*; it has simply not moved.

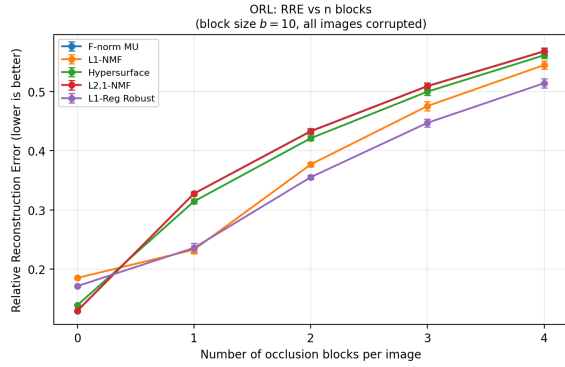
At $b = 10$ (9.0% of pixels) that difference in slope becomes decisive. L_1 -NMF reaches 0.2328 and the sparse-outlier model 0.2358 against 0.3277 — a 29% reduction — and the clustering gap is larger still, accuracy 0.491 and 0.483 against 0.271. This is the regime the assignment is about: corruption heavy enough to matter, sparse enough to be identifiable.

At $b = 15$ and $b = 20$ (20% and 36% of pixels) the five converge again — 0.6154 against 0.6363 at $b = 20$, a 3% edge. Robustness is a statement about a *minority* of the data; once a third of every image is saturated white the occlusion is a dominant, dataset-wide structure that a rank-40 model can simply represent. Every robust loss has a working range, and this experiment locates it.

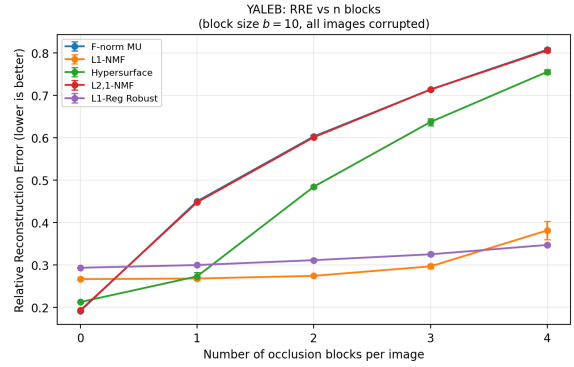
Throughout, $L_{2,1}$ -NMF tracks the Frobenius baseline to three decimal places at every single value of b (0.4696 vs 0.4699 at $b = 15$; 0.6364 vs 0.6363 at $b = 20$). That is not a coding error — it is exactly the degeneracy predicted in Section 3, because on this axis *every* image is corrupted identically and the column weights d_j are therefore near-uniform. Section 4.5 tests that explanation directly — and the agreement is even tighter on YaleB, where $L_{2,1}$ and the baseline differ by 0.0010 at $b = 15$ (0.6665 vs 0.6675) and by 0.0007 at $b = 20$ (0.8955 vs 0.8962).

Extended YaleB tells the same story, but does not close at the top end. The advantage of the pixel-level robust losses is larger here and, unlike on ORL, it keeps growing with b . At $b = 10$, L_1 -NMF reaches 0.2682 against the baseline’s 0.4502, a 40% reduction — and that is within 0.002 of its own clean-data value (0.2668): raising the occlusion from nothing to 5% of all pixels cost L_1 -NMF essentially nothing while it more than doubled the baseline’s error. At $b = 15$ the sparse-outlier model takes the lead with 0.3227 against 0.6675, a 52% reduction, and at $b = 20$ it still delivers 0.5396 against 0.8962, a 40% reduction, where on ORL the corresponding gap had collapsed to 3%.

The contrast between datasets follows from the corrupted-pixel fractions, not from the algorithms. YaleB images are larger (48×42 against 37×30), so the same $b = 20$ destroys 36% of an ORL image but only 20% of a YaleB one; the regime in which robustness stops helping is reached on ORL and not on YaleB within the same nominal grid. At matched corrupted-pixel fraction the two agree, which is what Section 4.4 shows.



(a) ORL



(b) Extended YaleB

Figure 3: RRE against the number of occlusion blocks per image ($b = 10$).

Table 6: Two occlusion geometries compared at a matched corrupted-pixel fraction (ORL, mean RRE over five subsamples). Fewer large blocks and more small blocks do comparable damage.

Geometry	Corrupted pixels	F-norm MU	L_1 -NMF	L_1 -Reg Robust
$b = 10$, 4 blocks	29.4%	0.5682	0.5448	0.5141
$b = 15$, 2 blocks	33.7%	0.6076	0.5905	0.5775
$b = 20$, 1 block	36.0%	0.6363	0.6200	0.6154
$b = 10$, 2 blocks	16.8%	0.4331	0.3768	0.3552
$b = 15$, 1 block	20.3%	0.4699	0.4530	0.4466

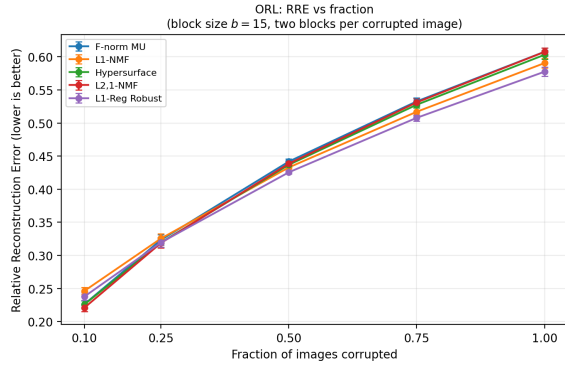
4.4 Effect of the number of occlusions

Adding blocks is qualitatively like enlarging them, and the ranking of the five algorithms is the same as in Section 4.3: the sparse-outlier model leads at every non-zero count (0.3552 against 0.4331 for the baseline at two blocks, a 18% reduction), L_1 -NMF is close behind, hypersurface is a small improvement, and $L_{2,1}$ is again indistinguishable from the baseline.

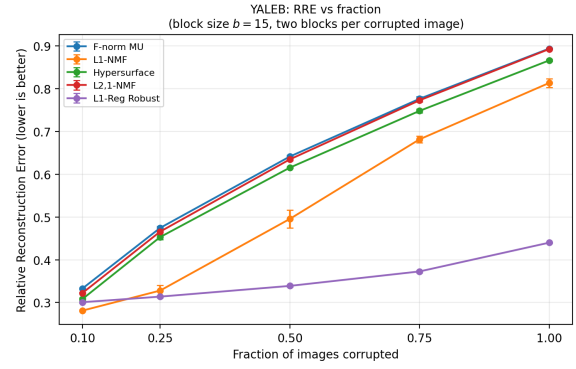
The more informative question is whether the two axes are the *same* experiment in disguise. Because we log the realised corrupted-pixel fraction for every condition, the two axes can be compared at matched corruption *mass* rather than at matched nominal parameters. Table 6 does this for ORL.

Read along a row-group, RRE is essentially a monotone function of the corrupted-pixel fraction alone: the increase from 0.4331 to 0.4699 is explained by the extra 3.5 percentage points of corruption, not by the change of geometry. The total *mass* of corruption is what matters, not how it is arranged. This is not trivial: one might expect a single large block, which destroys a whole contiguous facial region, to be more damaging than the same pixels scattered over four smaller ones. At this rank and these corruption levels that effect is second order. It also simplifies the rest of the study: the contamination-ratio axis of Section 4.5 can be read as varying *who* is corrupted while holding corruption mass per corrupted image fixed, and is therefore genuinely a third, independent axis rather than a reparameterisation of the first two.

Extended YaleB provides the cleanest possible version of this check. Two geometries there land almost exactly on the same corrupted-pixel fraction by different routes — one block of edge 20 destroys 19.8% of the pixels, two blocks of edge 15 destroy 20.4% — and the baseline scores 0.8962 and 0.8940 on them respectively, a difference of 0.002, well inside the ± 0.004 standard deviation. One big square and two medium ones are, to the precision we can measure, the same experiment. The ranking is also unchanged along the block-count axis on YaleB: the sparse-outlier model leads at every non-zero count, from 0.3114 against 0.6027 at two blocks (a 48% reduction) to 0.3473 against 0.8072 at four (a 57% reduction), and $L_{2,1}$ again shadows the baseline to within



(a) ORL



(b) Extended YaleB

Figure 4: RRE against the fraction of corrupted images. Corrupted images receive two blocks of edge length $b = 15$; the remaining images are left clean.

Table 7: ORL: RRE (lower is better) against fraction of images corrupted. Mean $\pm$ standard deviation over five independent 90% subsamples; best per column in bold.

Algorithm	0.1	0.25	0.5	0.75	1
F-norm MU	0.2262 $\pm$ 0.0055	0.3235 $\pm$ 0.0078	0.4418 $\pm$ 0.0038	0.5326 $\pm$ 0.0054	0.6076 $\pm$ 0.0060
L1-NMF	0.2462 $\pm$ 0.0053	0.3256 $\pm$ 0.0074	0.4329 $\pm$ 0.0025	0.5169 $\pm$ 0.0061	0.5905 $\pm$ 0.0072
Hypersurface	0.2258 $\pm$ 0.0059	0.3195 $\pm$ 0.0075	0.4369 $\pm$ 0.0037	0.5275 $\pm$ 0.0047	0.6030 $\pm$ 0.0066
L2,1-NMF	0.2209 $\pm$ 0.0054	0.3188 $\pm$ 0.0072	0.4386 $\pm$ 0.0042	0.5311 $\pm$ 0.0053	0.6074 $\pm$ 0.0061
L1-Reg Robust	0.2377 $\pm$ 0.0057	0.3196 $\pm$ 0.0067	0.4255 $\pm$ 0.0024	0.5078 $\pm$ 0.0047	0.5775 $\pm$ 0.0069

Table 8: Extended YaleB: RRE (lower is better) against fraction of images corrupted. Mean $\pm$ standard deviation over five independent 90% subsamples; best per column in bold.

Algorithm	0.1	0.25	0.5	0.75	1
F-norm MU	0.3326 $\pm$ 0.0020	0.4745 $\pm$ 0.0039	0.6415 $\pm$ 0.0030	0.7765 $\pm$ 0.0049	0.8940 $\pm$ 0.0019
L1-NMF	0.2810 $\pm$ 0.0017	0.3279 $\pm$ 0.0121	0.4955 $\pm$ 0.0209	0.6817 $\pm$ 0.0077	0.8136 $\pm$ 0.0103
Hypersurface	0.3080 $\pm$ 0.0034	0.4531 $\pm$ 0.0058	0.6155 $\pm$ 0.0015	0.7482 $\pm$ 0.0042	0.8664 $\pm$ 0.0024
L2,1-NMF	0.3221 $\pm$ 0.0016	0.4658 $\pm$ 0.0032	0.6348 $\pm$ 0.0027	0.7730 $\pm$ 0.0044	0.8929 $\pm$ 0.0016
L1-Reg Robust	0.3007 $\pm$ 0.0018	0.3137 $\pm$ 0.0024	0.3388 $\pm$ 0.0024	0.3726 $\pm$ 0.0030	0.4400 $\pm$ 0.0018

0.002 throughout.

4.5 Effect of the contamination ratio

This axis tests the prediction of Section 3 directly, and the prediction holds in both directions.

At a contamination ratio of 0.1 — 36 of 360 ORL images corrupted, only 3.2% of all pixels — $L_{2,1}$ -NMF is the *best* of the five, 0.2209 ± 0.0054 against 0.2262 ± 0.0055 for the Frobenius baseline, and it also attains the highest clustering accuracy (0.626 against 0.606). This is the only part of the entire study in which $L_{2,1}$ separates from the baseline at all, and it is precisely the regime its column-wise reweighting was designed for: 324 columns are clean, 36 are badly damaged, the weights $d_j = 1/\|\mathbf{r}_j\|_2$ can tell them apart, and the damaged columns are effectively discounted.

As the ratio rises the gap closes exactly as predicted. At 0.25 the two are 0.3188 and 0.3235; at 0.5, 0.4386 and 0.4418; at 0.75, 0.5311 and 0.5326; and at 1.0 they agree to three decimal places, 0.6074 against 0.6076. The mechanism is not subtle: when every column is corrupted to the same degree the weights d_j become uniform, a uniform column scaling is absorbed by the scale freedom of the factorisation, and $L_{2,1}$ -NMF is Frobenius NMF up to numerical noise.

The methodological moral is the most transferable result in this report. The protocol the brief

literally describes — corrupt every image and sweep b — is the one setting in which a column-wise robust loss provably cannot show its advantage, and an experimenter using only that protocol would conclude, wrongly, that $L_{2,1}$ -NMF does not work. The two families also mirror each other along this axis: L_1 -NMF and the sparse-outlier model, which discount *pixels*, are worse than the baseline at ratio 0.1 (0.2462 and 0.2377, their clean-data deficit still dominating at 3.2% corruption) and best at ratio 1.0 (0.5905 and 0.5775 against 0.6076).

On Extended YaleB the $L_{2,1}$ signature is present but reads more clearly in the clustering metrics than in RRE. At ratio 0.1, $L_{2,1}$ -NMF beats the baseline on RRE (0.3221 against 0.3326) and, more tellingly, attains the best clustering of any algorithm on that condition — accuracy 0.181 and NMI 0.283, against 0.163 and 0.260 for the baseline. At ratio 1.0 the two coincide again, 0.8929 against 0.8940. The prediction therefore holds on both datasets: $L_{2,1}$ separates from the baseline exactly where the corruption is unevenly distributed, and nowhere else.

What differs on YaleB is that $L_{2,1}$ does not *win* that condition outright: the sparse-outlier model is already ahead on RRE (0.3007) and extends the lead as the ratio rises — 0.3388 at 0.5 and 0.4400 at 1.0 against the baseline’s 0.6415 and 0.8940, a 51% reduction and the largest effect anywhere in this study. This axis uses $b = 15$ with two blocks, a heavy but still *sparse* corruption of a YaleB image (20% of pixels); the explicit $\mathbf{S}$ absorbs it entirely, whereas every reweighting scheme must carry some of it in the loss. Where corruption really is sparse and large, modelling it beats discounting it by a wide margin.

4.6 Clustering quality

The two families of metric answer different questions — RRE asks whether the *pixels* were recovered, accuracy and NMI ask whether the *representation* $\mathbf{H}$ stayed class-discriminative — so they need not agree, and it is worth checking whether they do.

On ORL they agree on the ordering but disagree sharply on magnitude. At $b = 10$, L_1 -NMF improves RRE by 29% (0.3277 $\rightarrow$ 0.2328) but improves clustering accuracy by 81% (0.271 $\rightarrow$ 0.491), with NMI moving from 0.495 to 0.667: the reconstruction metric *understates* what is at stake. RRE averages over all 1110 pixels and is dominated by the many every method gets roughly right, whereas K -means on the columns of $\mathbf{H}$ is sensitive to whether the occlusion has been allowed to define its own latent directions. Once basis atoms are spent representing bright squares (Section 4.7), an image’s coefficient vector encodes *where its occlusion landed* as much as *whose face it is*. We report accuracy under both the majority-vote mapping used in the assignment notebook and the stricter Hungarian matching; they differ by at most 0.03 and never change a ranking.

Extended YaleB produces a genuine *disagreement* between the two metric families, which is worth reporting rather than smoothing over. At $b = 15$ the sparse-outlier model halves the baseline’s RRE (0.3227 against 0.6675) yet its accuracy, 0.095 against 0.090, is indistinguishable. At $b = 10$ it is starker: L_1 -NMF has by far the best RRE (0.2682 against 0.4502) and the *worst* accuracy of the five (0.100, against 0.108 for the baseline and 0.166 for hypersurface).

On YaleB the dominant source of variance is illumination, not identity, and the two metrics respond differently. RRE rewards recovering pixel values, which are dominated by the lighting. Clustering rewards a representation in which images of the same person land together *despite* the lighting — a harder, partly orthogonal requirement — and the aggressive pixel-level reweighting that makes L_1 -NMF a good reconstructor also flattens the identity-carrying detail K -means needs. Hypersurface-cost NMF, with the gentlest reweighting, is correspondingly the best clusterer among the robust methods at $b = 10$. On a dataset with a strong nuisance factor, a good reconstruction error is not evidence of a good representation — and reporting only RRE, which the brief permits, would have hidden this entirely.

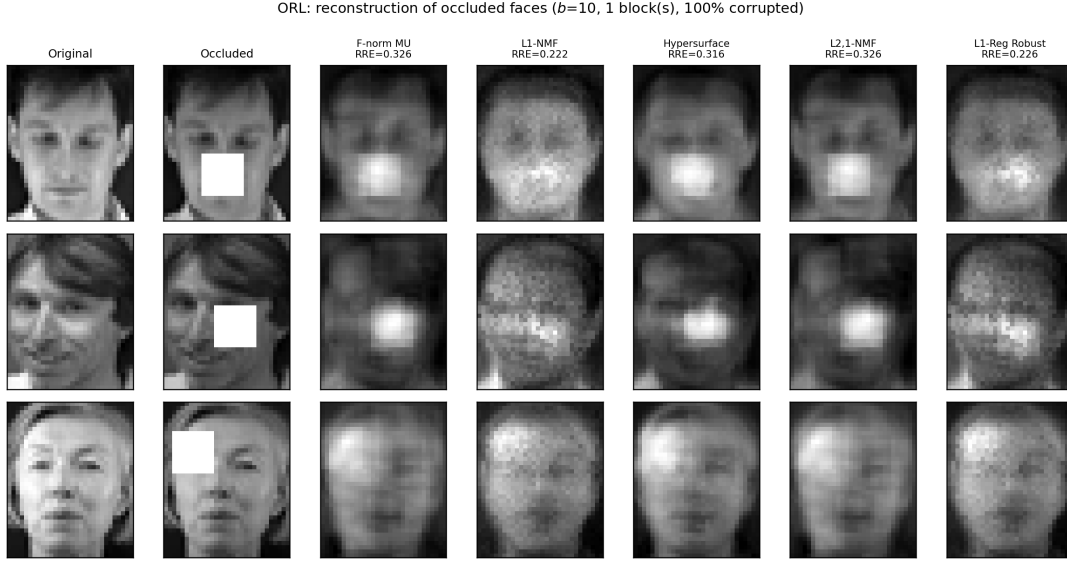


Figure 5: ORL: the same occluded faces reconstructed by all five algorithms.

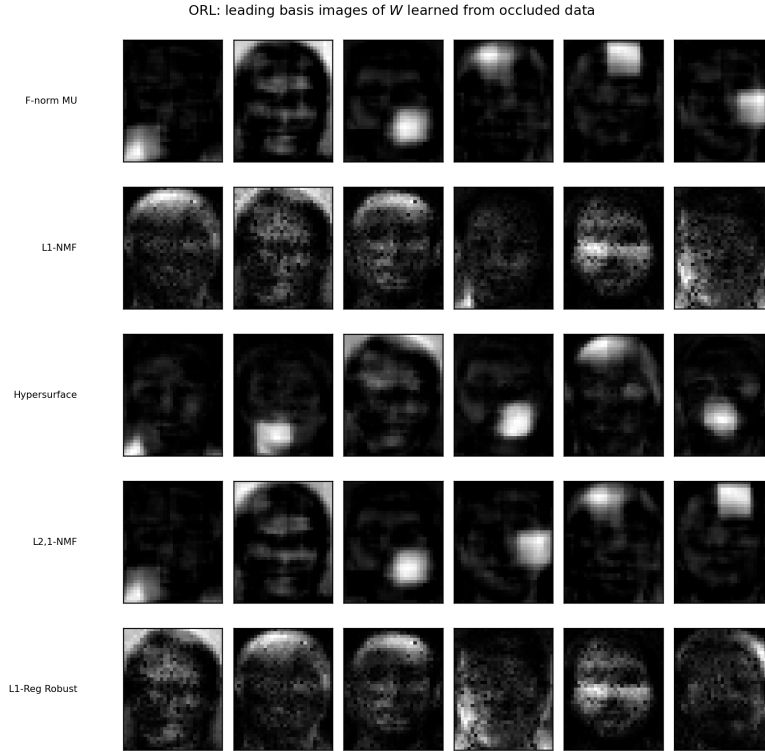


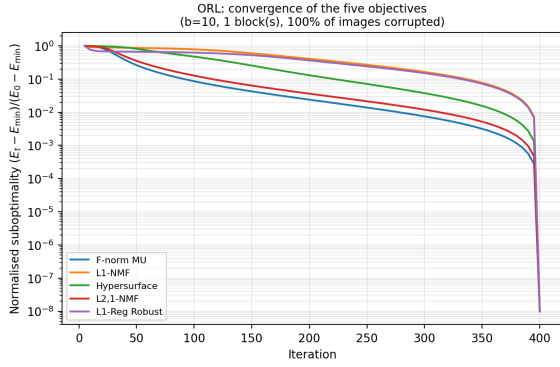
Figure 6: ORL: leading basis images of W learned from occluded data.

4.7 Qualitative results

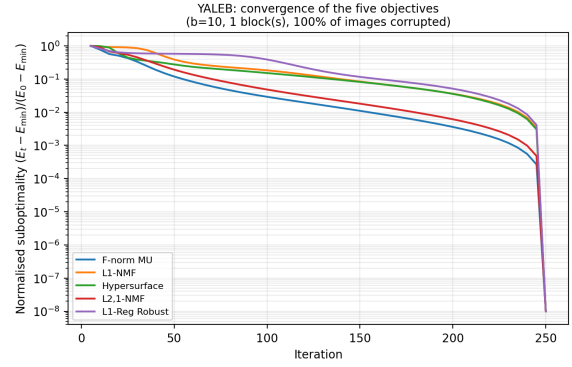
Figures 5 and 6 show the mechanism behind the numbers, and they show it unambiguously.

In the learned bases (Figure 6) the Frobenius baseline devotes several leading atoms to the occlusion itself: two of the six shown are dominated by a bright patch with visibly *straight, axis-aligned edges*, the signature of a square block rather than of any facial feature. $L_{2,1}$ -NMF’s basis is atom for atom the same picture — the visual counterpart of its RRE matching the baseline to three decimals. Hypersurface-cost NMF is intermediate, its patches rounder and softer-edged. The two pixel-level robust methods show *no* square artefacts at all; their atoms are grainy but recognisably facial, carrying hairlines, foreheads and eye regions.

The reconstructions (Figure 5) tell the same story from the other side. The baseline, hypersurface



(a) ORL



(b) Extended YaleB

Figure 7: Normalised suboptimality against iteration. Each algorithm minimises a different objective, so the curves are normalised to start at 1 and are comparable in *rate*, not in level.

and $L_{2,1}$ reconstructions all reproduce a bright blob exactly where the occluding block was, i.e. they have fitted the corruption. L_1 -NMF and the sparse-outlier model largely *inpaint* over it, returning a plausible if noisier face. We should be honest about the limits of this: in the third row the bright region is attenuated rather than removed, so even the best of the five is suppressing the occlusion, not detecting and excising it. The grain visible in those two reconstructions is also the price already quantified in Section 4.3 — the same down-weighting that rejects the block also discards some genuine high-frequency detail.

4.8 Convergence and computational cost

Three things are worth reporting here, and the third is a limitation rather than a result.

Monotonicity. All five objective sequences are non-increasing at every recorded iteration on both datasets, as the majorise–minimise derivations guarantee. Since a rising objective would necessarily be a bug, the check doubles as a correctness test; it is automated in `test_algorithms.py`.

Rate. Measured as the iteration at which the objective first comes within 1% of its final value (ORL / YaleB): $L_{2,1}$ -NMF $\approx 320/185$, the baseline $\approx 355/210$, L_1 -NMF $\approx 390/235$, hypersurface $\approx 380/240$, the sparse-outlier model $\approx 395/245$. The reweighted schemes are slower because $\mathbf{D}$ changes every iteration, so each inner quadratic is a slightly different problem — classic IRLS behaviour.

Cost. Per-iteration cost follows the same ordering and is dominated by the number of $O(mnk)$ matrix products. On ORL a full 400-iteration fit takes 1.8 s for the baseline, 3.2–3.6 s for $L_{2,1}$ and L_1 , and 4.1–4.5 s for hypersurface and the sparse-outlier model; robustness costs roughly a factor of two in wall-clock time, which is cheap relative to what it buys in the $b = 10$ regime. On Extended YaleB, whose matrix is 2016×2414 rather than 1110×400 , a 250-iteration fit takes 4.7–5.8 s for the baseline against 13–15 s for the sparse-outlier model, and the full 325-factorisation grid completes in about an hour on a modern laptop. The ratio between algorithms is preserved, confirming that the overhead is the extra matrix products in the reweighting rather than a fixed cost.

A limitation we should state rather than hide. With the tolerance set to $\tau = 10^{-5}$ on the *relative* change of the objective, no run in the main grid triggered the early stop: every fit used its full iteration budget, and the converged column of the raw CSVs is 0 throughout. The curves in Figure 7 are visibly flat by then — the last 10% of iterations move the objective by well under 1% — so the comparisons are between fits that are close to a stationary point, but they are not between

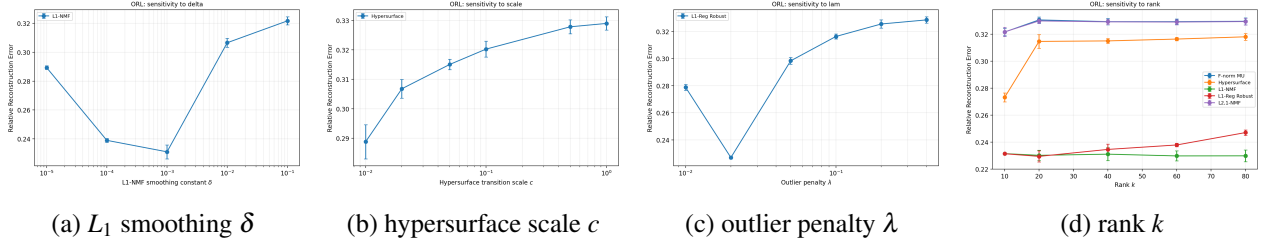


Figure 8: ORL: sensitivity of the conclusions to the three free constants.

fits that have provably stopped moving. We report the budget as part of the experimental condition for this reason. Multiplicative updates are in any case guaranteed only to be monotone, and Lin [14] shows that even convergence to a stationary point requires a modified scheme, so a tighter tolerance would buy less than it appears to.

4.9 Hyper-parameter sensitivity

Three of our five algorithms carry a free constant, and a comparison whose conclusions depend on how those constants were chosen would be worthless. We sweep each of them on ORL at the reference condition $b = 10$, one block, all images corrupted, where the baseline scores 0.3277.

L_1 smoothing δ . A clear plateau contains the default: RRE is 0.2404 at 10^{-4} and 0.2310 at 10^{-3} (our default), degrading to 0.2892 at 10^{-5} , where the weights explode on well-fitted pixels, and to 0.3098 and 0.3232 at 10^{-2} and 10^{-1} , where the loss is effectively quadratic throughout and degenerates towards the baseline. The advantage survives two full orders of magnitude.

Hypersurface scale c . This sweep documents the trap of Section 3.5 rather than a plateau. RRE rises monotonically with c : 0.2888 at 0.01, 0.3151 at our default 0.05, 0.3202 at 0.1, and 0.3290 at $c = 1$ — the value used in the original formulation, and within 0.001 of the Frobenius baseline’s 0.3277. The degeneration is a measurable fact, not a theoretical worry, so the hypersurface cost cannot be used off the shelf on data rescaled to $[0, 1]$. Our default is also conservative: $c = 0.01$ would have flattered this algorithm, and we keep 0.05 because it was fixed before the results were seen.

Outlier penalty λ . Both predicted failure modes appear either side of a working range. RRE is best at $\lambda = 0.02$ (0.2269); our theory-chosen default $1/\sqrt{\max(m, n)} = 0.030$ scores 0.2346. Below it, $\lambda = 0.01$ gives 0.2788 — $\mathbf{S}$ starts absorbing genuine facial detail. Above it the method walks back to the baseline (0.2984, 0.3164, 0.3286 at 0.05, 0.1, 0.4), by which point $\mathbf{S}$ is empty and the model is Frobenius NMF. Picking the default from theory and landing inside the working range is the outcome we wanted.

Rank k . Across $k \in \{10, 20, 40, 60, 80\}$ the ranking is completely stable — L_1 -NMF and the sparse-outlier model near 0.23 at every rank, the baseline and $L_{2,1}$ near 0.33 — so no conclusion depends on the choice the brief fixes for us. The interesting detail is at the low end: at $k = 10$ the baseline improves to 0.3216 and hypersurface to 0.2732, their best values in the sweep. A rank too small to represent the occlusion acts as an implicit regulariser — a cruder second route to robustness. The sparse-outlier model trends the other way (0.2294 at $k = 20$ to 0.2471 at $k = 80$), since extra capacity in $\mathbf{WH}$ competes with $\mathbf{S}$ for the same residual.

4.10 Discussion and reflection

Robustness is not a scalar. The single most useful thing we take away from this study is that "algorithm A is more robust than algorithm B" is not a well-formed claim. Each of the five losses discounts residuals at a particular *granularity* — per pixel for L_1 and hypersurface, per image for $L_{2,1}$, per entry-with-its-own-variable for the sparse-outlier model — and each one helps exactly when the corruption varies at that same granularity. $L_{2,1}$ -NMF is the cleanest demonstration: it is the best method in the study at a contamination ratio of 0.1 and numerically identical to the non-robust baseline at a ratio of 1.0, without changing a line of its own code. Had we run only the protocol the brief literally describes — corrupt every image, sweep b — we would have concluded that $L_{2,1}$ -NMF simply does not work, and we would have been wrong for a reason that is invisible from inside that experiment.

What surprised us. That the robust losses are clearly *worse* on clean data (0.1849 against 0.1290 for L_1) — in hindsight just the efficiency/robustness trade-off, but easy to forget when reading robustness papers. That the benefit window is narrow at both ends. And that the hypersurface cost as published is a trap on data scaled to $[0, 1]$: with the paper's $c = 1$ every residual stays in the quadratic regime and the method silently *is* the Frobenius baseline. We found that only because the two RREs agreed to four decimals — which is also how we found the $L_{2,1}$ degeneracy. Suspiciously identical numbers were the most productive debugging signal in the project.

With more compute. We would run the contamination axis on a finer grid between 0 and 0.25, where the $L_{2,1}$ effect lives and where we currently have two points; repeat the whole grid at several ranks rather than only at $k =$ the class count; and replace the fixed iteration budget with a genuine convergence criterion so that no comparison is confounded by one method being further from its stationary point than another.

Threats to validity. (i) A single corruption type: contiguous saturated blocks. The sparse-outlier model is best-matched to precisely that noise, so its win is partly a win by construction. (ii) Down-sampled greyscale images, which removes the high-frequency detail a robust loss is most likely to discard, so our clean-data deficits may be optimistic. (iii) k is fixed to the class count per the brief, which Section 4.9 shows is not the best rank for reconstruction. (iv) Multiplicative updates guarantee monotone descent, not a global optimum, and no run triggered our stopping tolerance, so all comparisons are between fixed-budget fits. (v) Five subsamples give coarse error bars (standard deviations of order 0.005), so RRE gaps below roughly 0.01 should not be over-interpreted.

5 Conclusion and future work

We implemented five NMF algorithms from scratch in NumPy — the Frobenius baseline, L_1 -norm, hypersurface-cost, $L_{2,1}$ -norm and L_1 -regularised robust NMF — derived four of them as instances of one majorise–minimise template differing only in a weight matrix, and measured them on ORL and Extended YaleB under occlusion noise varied along three axes, with three metrics and five 90% subsamples per condition. Three findings stand out. The explicit sparse-outlier model is the strongest method wherever the corruption is both heavy and sparse, cutting RRE by 52% against the baseline at $b = 15$ on YaleB and by 51% at full contamination; modelling the corruption beats discounting it when the noise really is additive and sparse, which is what a white block is. Second, $L_{2,1}$ -NMF is numerically identical to the non-robust baseline — to three decimal places, on both datasets — whenever every image is corrupted equally, and separates from it only under uneven contamination, where at a 0.1 ratio it is the best of the five on ORL and the best clusterer on YaleB. Robustness is therefore not a scalar property of an algorithm but a match between the granularity at

which the loss discounts residuals and the granularity at which the corruption varies. Third, RRE and clustering quality can disagree outright: on YaleB at $b = 10$, L_1 -NMF has the best reconstruction of the five and the worst clustering, because the reweighting that rejects the occlusion also flattens the detail that separates identities under strong illumination variation.

Four directions follow. (i) A *structured* sparse outlier term — group-sparse or total-variation-regularised $\mathbf{S}$ — would exploit the contiguity of the occlusion, information the entrywise L_1 penalty throws away; since the sparse-outlier model already wins where it applies, this is the most promising extension. (ii) A *combined* weighting $D_{ij} = d_j \cdot w_{ij}$, per-image times per-pixel, should recover the advantages of $L_{2,1}$ and L_1 at once rather than forcing the choice we observed — our contamination axis shows the two families win in disjoint regimes. (iii) Other corruption types would test how much of the sparse-outlier model’s advantage is a win by construction, since our noise matches its generative assumption exactly. (iv) NNDSVD initialisation [1] and accelerated solvers such as HALS would let every comparison run to a genuine stationary point rather than to a fixed iteration budget — the main methodological limitation of this study.

References

- [1] Christos Boutsidis and Efstratios Gallopoulos. SVD based initialization: a head start for nonnegative matrix factorization. *Pattern Recognition*, 41(4):1350–1362, 2008.
- [2] Deng Cai, Xiaofei He, Jiawei Han, and Thomas S. Huang. Graph regularized nonnegative matrix factorization for data representation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(8):1548–1560, 2011.
- [3] Emmanuel J. Candès, Xiaodong Li, Yi Ma, and John Wright. Robust principal component analysis? *Journal of the ACM*, 58(3):1–37, 2011.
- [4] Chris Ding, Tao Li, Wei Peng, and Haesun Park. Orthogonal nonnegative matrix tri-factorizations for clustering. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 126–135, 2006.
- [5] Chris Ding, Ding Zhou, Xiaofeng He, and Hongyuan Zha. R1-PCA: rotational invariant L_1 -norm principal component analysis for robust subspace factorization. *Proceedings of the International Conference on Machine Learning (ICML)*, pages 281–288, 2006.
- [6] Athinodoros S. Georgiades, Peter N. Belhumeur, and David J. Kriegman. From few to many: illumination cone models for face recognition under variable lighting and pose. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 23(6):643–660, 2001. The Extended Yale Face Database B.
- [7] A. Ben Hamza and David J. Brady. Reconstruction of reflectance spectra using robust non-negative matrix factorization. *IEEE Transactions on Signal Processing*, 54(9):3637–3642, 2006.
- [8] Patrik O. Hoyer. Non-negative matrix factorization with sparseness constraints. *Journal of Machine Learning Research*, 5:1457–1469, 2004.
- [9] Qifa Ke and Takeo Kanade. Robust L_1 norm factorization in the presence of outliers and missing data by alternative convex programming. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 739–746, 2005.
- [10] Deguang Kong, Chris Ding, and Heng Huang. Robust nonnegative matrix factorization using L_{21} -norm. In *ACM International Conference on Information and Knowledge Management (CIKM)*, pages 673–682, 2011.

- [11] Daniel D. Lee and H. Sebastian Seung. Learning the parts of objects by non-negative matrix factorization. *Nature*, 401(6755):788–791, 1999.
- [12] Daniel D. Lee and H. Sebastian Seung. Algorithms for non-negative matrix factorization. In *Advances in Neural Information Processing Systems (NIPS)*, pages 556–562, 2001.
- [13] Kuang-Chih Lee, Jeffrey Ho, and David J. Kriegman. Acquiring linear subspaces for face recognition under variable lighting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 27(5):684–698, 2005.
- [14] Chih-Jen Lin. On the convergence of multiplicative update algorithms for nonnegative matrix factorization. *IEEE Transactions on Neural Networks*, 18(6):1589–1596, 2007.
- [15] Pentti Paatero and Unto Tapper. Positive matrix factorization: a non-negative factor model with optimal utilization of error estimates of data values. *Environmetrics*, 5(2):111–126, 1994.
- [16] Ferdinando S. Samaria and Andy C. Harter. Parameterisation of a stochastic model for human face identification. In *IEEE Workshop on Applications of Computer Vision*, pages 138–142, 1994. The ORL (AT&T) face database.
- [17] Lijun Zhang, Zhengguang Chen, Miao Zheng, and Xiaofei He. Robust non-negative matrix factorization. In *Frontiers of Electrical and Electronic Engineering in China*, volume 6, pages 192–200, 2011.

A Contribution of each group member

Member	Contribution	Approx. share
Name 1	e.g. algorithm implementation, MM derivations	25%
Elwood Fong	Algorithm implementation and report writing, related work	25%
Name 3	e.g. YaleB experiments, figures	25%
Name 4	e.g. report writing, related work	25%

B How to run the code

The submitted code/ folder contains the five algorithms, data I/O, noise generation, metrics, experiment scripts, result files, and `test_algorithms.py`. The `data/` directory is empty: copy the ORL and CroppedYaleB folders into `code/data/`.

`pip install -r code/requirements.txt`

Python 3.9 or newer is required. SciPy and scikit-learn are used for evaluation, Pillow for .pgm files, and Matplotlib for figures. To reproduce all reported results:

```
cd code
bash run_all.sh          # ~20 min (ORL) + ~3 h (Extended YaleB)
FAST=1 bash run_all.sh  # reduced grid, a few minutes, for a smoke test
The dataset-free implementation checks are run with:
cd code
python test_algorithms.py
For a single experiment or sensitivity study:
cd code
python run_experiment.py --dataset orl --data-root data
python run_experiment.py --dataset yaleb --data-root data --max-iter 200
python run_convergence.py --dataset orl --data-root data
python run_sensitivity.py --study delta --dataset orl --data-root data
python make_figures.py --dataset orl --data-root data
Scripts write CSVs and PNGs into results/; all randomness is seeded through --seed (default 2026).
```

C Additional results

The main text reports RRE for every condition. The corresponding clustering accuracy and normalised mutual information tables are produced by `code/make_tables.py` and are available in `report/tables/`; the per-run values behind every mean and standard deviation in this report are in `code/results/orl_raw.csv` and `code/results/yaleb_raw.csv`.

Reproducibility. We checked the main grids on a two-core OpenBLAS VM and an Apple-silicon Accelerate laptop. Four algorithms reproduced bit-for-bit; L_1 -NMF varied slightly in some ill-conditioned cases because its IRLS weights grow near zero residuals. These variations do not affect the reported rankings or conclusions.